

Checklist Exhaustive - Projet Chef d'Œuvre (E1 à E5)

Cette liste reprend l'intégralité des critères d'évaluation présents dans mes documents de référence. Chaque case doit être cochée pour garantir la validation du diplôme.

Checklist Exhaustive - Projet Chef d'Œuvre (E1 à E5)	0
■ BLOC E1 : Gérer des données de l'information (Data)	1
A1. Programmer la collecte de données	1
A2. Développer la mise à disposition technique des données	2
■ BLOC E2 : Intégrer des services d'IA (Veille & SaaS)	3
A3. Accompagner le choix et l'intégration d'un service IA	3
■ BLOC E3 : Déployer un modèle d'IA (MLOps)	4
A4. Réaliser l'intégration d'un modèle IA	4
A5. Déploiement MLOps	4
■ BLOC E4 : Réaliser une application (Dev App)	6
A6. Concevoir l'application	6
A7. Développer interfaces et fonctionnalités	6
A8. Tests et Contrôle	7
■ BLOC E5 : Maintenance et Monitoring	8
A9. Maintien en condition opérationnelle	8
Checklist des Ressources & Stack Technique à utiliser pour le projet	9
1. Gestion de Projet & Méthodologie	9
2. Développement & Code	9
3. Data (Bloc E1)	9
4. Intelligence Artificielle (Blocs E2 & E3)	9
5. Backend & API	10
6. Frontend (Application cliente)	10
7. DevOps & Infrastructure (Blocs E4 & E5)	10

BLOC E1 : Gérer des données de l'information (Data)

A1. Programmer la collecte de données

C1. Automatiser l'extraction de données

- ☐ J'ai identifié les contraintes techniques propres aux sources de données (documentation, confidentialité, etc.).
- ☐ J'ai rédigé les spécifications techniques pour l'extraction des données.
- ☐ J'ai construit les requêtes HTTP pour récupérer des données depuis un service web (API REST).
- ☐ J'ai programmé la lecture d'un fichier de données dans un script.
- ☐ J'ai programmé le téléchargement du HTML d'une ou plusieurs pages web (Scraping).
- ☐ J'ai établi une connexion programmatique à un Système de Gestion de Base de Données (SGBD).
- ☐ J'ai établi une connexion programmatique à un système Big Data (Hive, Impala, etc.).
- ☐ J'ai programmé le filtrage/parsing des données depuis l'API.
- ☐ J'ai programmé le filtrage/parsing des données depuis les fichiers.
- ☐ J'ai programmé le filtrage/parsing des données depuis le HTML (Scraping).
- ☐ J'ai exécuté des requêtes d'extraction de type SQL via le script.
- ☐ J'ai exécuté des requêtes d'extraction depuis le système Big Data via le script.
- ☐ Le script est fonctionnel et récupère toutes les données visées.
- ☐ Le script est structuré (initialisation, traitement, gestion erreurs, sauvegarde).
- ☐ Le script est versionné et accessible sur un dépôt Git distant.

C2. Développer des requêtes SQL d'extraction

- ☐ J'ai écrit les requêtes SQL pour extraire les données de la base de données relationnelle.
- ☐ J'ai écrit les requêtes pour extraire les données du système Big Data.
- ☐ Les requêtes sont fonctionnelles et extraient exactement les données visées.
- ☐ J'ai documenté les requêtes (choix de sélections, filtrages, jointures).
- ☐ La documentation explicite les optimisations appliquées aux requêtes.

C3. Agrégation et nettoyage des données

- ☐ J'ai rédigé les spécifications techniques pour l'agrégation.
- ☐ J'ai programmé l'agrégation des données de toutes les sources en un jeu unique.
- ☐ J'ai programmé l'identification des entrées corrompues (partielles/manquantes).
- ☐ J'ai programmé la suppression des entrées corrompues.

- ☐ J'ai programmé l'identification des formats non normalisés.
- ☐ J'ai programmé l'homogénéisation des formats (dates, unités, etc.).
- ☐ Le script produit un jeu de données final unique, nettoyé et normalisé.
- ☐ Le script est versionné sur Git.
- ☐ La documentation du script est complète (dépendances, logique algo, choix de nettoyage).

A2. Développer la mise à disposition technique des données

C4. Création de la base de données (et RGPD)

- ☐ J'ai rédigé les spécifications techniques pour le stockage.
- ☐ J'ai modélisé la structure des données selon la méthode Merise (MCD/MLD).
- ☐ Le modèle physique est fonctionnel et intégré sans erreur.
- ☐ J'ai choisi un SGBD adapté aux contraintes.
- ☐ J'ai créé la base de données.
- ☐ J'ai documenté la procédure d'installation du SGBD.
- ☐ J'ai rédigé/mis à jour le registre des traitements de données personnelles (RGPD).
- ☐ J'ai rédigé les procédures de tri des données personnelles (détection, suppression, conformité RGPD).
- ☐ J'ai programmé le script d'import des données dans la base.
- ☐ Le script d'import est fonctionnel.
- ☐ Le script d'import est versionné et documenté (dépendances, commandes).

C5. Développer une API de mise à disposition (REST)

- ☐ J'ai rédigé les spécifications techniques de l'API et de l'accès aux données.
- ☐ J'ai configuré les accès aux données depuis le serveur API.
- ☐ J'ai développé la réception et la validation des requêtes client.
- ☐ J'ai développé les requêtes BDD déclenchées par l'API.
- ☐ J'ai développé les réponses de l'API au client.
- ☐ J'ai développé les règles d'autorisation/accès aux endpoints.
- ☐ J'ai sécurisé l'API (ex: Top 10 OWASP).
- ☐ L'API est fonctionnelle (accès et mise à disposition des données).
- ☐ La documentation technique couvre tous les endpoints.
- ☐ La documentation couvre l'authentification/autorisation.
- ☐ La documentation respecte les standards (ex: OpenAPI/Swagger).

BLOC E2 : Intégrer des services d'IA (Veille & SaaS)

A3. Accompagner le choix et l'intégration d'un service IA

C6. Veille technique et réglementaire

- ☐ J'ai défini la/les thématiques de veille (liées au projet).
- ☐ J'ai planifié des temps de veille réguliers (ex: hebdo).
- ☐ J'ai choisi un outil d'agrégation de flux (RSS, newsletter, etc.).
- ☐ J'ai choisi un outil de partage/communication des synthèses.
- ☐ J'ai identifié des sources fiables (dont accessibilité, sécurité, data).
- ☐ J'ai qualifié la fiabilité des sources.
- ☐ J'ai configuré les outils d'agrégation.
- ☐ J'ai rédigé des synthèses des informations collectées.
- ☐ J'ai communiqué les synthèses aux parties prenantes (format accessible).

C7. Identifier des services IA (Benchmark)

- ☐ J'ai reformulé le besoin (problématique technique/fonctionnelle).
- ☐ J'ai identifié les contraintes (moyens, techniques, ops).
- ☐ J'ai réalisé un benchmark des services existants (étudiés vs non étudiés).
- ☐ J'ai explicité les raisons d'exclusion des services.
- ☐ Le benchmark détaille l'adéquation fonctionnelle.
- ☐ Le benchmark détaille la démarche éco-responsable des services.
- ☐ Le benchmark détaille les contraintes techniques/pré-requis.
- ☐ J'ai rédigé des conclusions préconisant une ou plusieurs solutions.

C8. Paramétrer un service IA

- ☐ J'ai créé l'environnement d'exécution (Compte SaaS, VPS, etc.).
- ☐ J'ai installé/configuré les dépendances (SDK, outils).
- ☐ J'ai géré les accès (comptes, groupes, droits).
- ☐ J'ai installé/configuré les outils de monitoring du service.
- ☐ Le service est accessible et authentifié.
- ☐ Le service répond aux besoins fonctionnels.
- ☐ Le monitoring est opérationnel.
- ☐ La documentation technique couvre installation, accès, dépendances.
- ☐ La documentation est accessible (normes handicap).

BLOC E3 : Déployer un modèle d'IA (MLOps)

A4. Réaliser l'intégration d'un modèle IA

C9. Développer une API exposant un modèle IA

- ☐ J'ai analysé les specs fonctionnelles/techniques.
- ☐ J'ai conçu l'architecture de l'API (endpoints, règles).
- ☐ J'ai choisi les outils/langages.
- ☐ J'ai installé l'environnement de développement.
- ☐ L'API vérifie et transforme les paramètres envoyés par le client.
- ☐ L'API exécute le modèle à partir de la requête.
- ☐ L'API renvoie le résultat au client.
- ☐ J'ai développé l'authentification/autorisation.
- ☐ J'ai sécurisé l'API (OWASP).
- ☐ J'ai développé des tests d'intégration pour les endpoints.
- ☐ Les tests passent sans bug.
- ☐ Le code est versionné sur Git distant.
- ☐ La documentation (OpenAPI) est rédigée et accessible.

C10. Intégrer l'API IA dans une application

- ☐ J'ai installé l'environnement de l'application cliente.
- ☐ J'ai programmé l'authentification vers l'API IA.
- ☐ J'ai programmé la communication avec les endpoints.
- ☐ J'ai intégré les adaptations d'interface nécessaires.
- ☐ J'ai testé l'accessibilité des interfaces modifiées.
- ☐ J'ai développé des tests d'intégration (côté app) sur le périmètre API.
- ☐ Les tests s'exécutent en totalité sans erreur.
- ☐ Le code est versionné sur le Git de l'application.

A5. Déploiement MLOps

C11. Monitorer le modèle IA

- ☐ J'ai listé les métriques à monitorer (perf modèle, stabilité data, santé système).
- ☐ J'ai défini les déclencheurs de réentraînement.
- ☐ J'ai choisi l'outil de monitoring et de consolidation.
- ☐ J'ai intégré les collecteurs de données.
- ☐ J'ai intégré l'outil de restitution (Dashboard type Grafana/Streamlit).
- ☐ J'ai configuré les alertes (email, notif).
- ☐ L'outil de restitution respecte les enjeux d'accessibilité.
- ☐ La chaîne de monitoring est testée et fonctionnelle.

- ☐ Code versionné sur Git.
- ☐ Documentation technique (install/maintenance) et utilisateur rédigée.

C12. Tests automatisés du modèle

- ☐ J'ai défini le périmètre des tests (format data, entraînement, éval).
- ☐ J'ai choisi les outils de test (ex: unittest, pytest).
- ☐ J'ai configuré l'environnement de test.
- ☐ J'ai intégré les tests (assertions, mocks, fixtures).
- ☐ Les tests couvrent le périmètre défini.
- ☐ Code et Données (si possible DVC) versionnés.
- ☐ Documentation technique des tests rédigée.

C13. Chaîne de livraison continue (CI/CD pour IA)

- ☐ J'ai défini étapes, tâches et déclencheurs (Pipeline).
- ☐ J'ai paramétré la chaîne (variables d'env, versions).
- ☐ J'ai intégré l'étape de test des données.
- ☐ J'ai intégré l'étape de test/entraînement/validation du modèle.
- ☐ J'ai intégré la génération de rapports (matrix confusion, accuracy) dans la livraison.
- ☐ J'ai intégré l'étape de livraison (ex: Pull Request automatique).
- ☐ Les fichiers de config CI/CD sont reconnus et exécutés.
- ☐ Code versionné sur Git.
- ☐ Documentation de la chaîne CI/CD rédigée et accessible.

BLOC E4 : Réaliser une application (Dev App)

A6. Concevoir l'application

C14. Analyser le besoin

- ☐ La modélisation des données respecte un formalisme (Merise/Entité-Relation).
- ☐ La modélisation des parcours utilisateurs respecte un formalisme (Wireframes/Schéma).
- ☐ Les User Stories couvrent contexte, scénarios et critères de validation.
- ☐ Les objectifs d'accessibilité sont intégrés aux critères d'acceptation (User Stories).
- ☐ Les objectifs d'accessibilité s'appuient sur un standard (WCAG/RG2AA).

C15. Concevoir le cadre technique

- ☐ Les specs techniques couvrent architecture, dépendances, environnement.
- ☐ Les choix techniques favorisent l'éco-responsabilité (Green IT).
- ☐ Les flux de données sont représentés (Diagramme de flux).
- ☐ Une Preuve de Concept (POC) est réalisée.
- ☐ La POC est accessible et fonctionnelle en pré-prod.
- ☐ La conclusion de la POC permet de décider de la poursuite du projet.

A7. Développer interfaces et fonctionnalités

C16. Coordonner la réalisation (Agile)

- ☐ Les cycles, rôles et rituels Agile sont respectés.
- ☐ Les outils de pilotage (Kanban, Backlog) sont disponibles et à jour.
- ☐ Les objectifs des rituels sont partagés.
- ☐ Les éléments de pilotage sont accessibles à tous.

C17. Développer composants et interfaces

- ☐ L'environnement de dev respecte les specs.
- ☐ Les interfaces respectent les maquettes.
- ☐ Les comportements (validation, navigation) respectent les specs.
- ☐ Les composants métier fonctionnent comme prévu.
- ☐ La gestion des droits/accès est développée.
- ☐ Les développements respectent l'éco-conception.
- ☐ Le Top 10 OWASP est implémenté.
- ☐ Des tests unitaires/intégration couvrent le métier et les accès.
- ☐ Sources versionnées sur Git.
- ☐ Documentation technique (install, arch, tests) rédigée et accessible.

A8. Tests et Contrôle

C18. Automatiser les tests (CI)

- ☐ L'outil de CI est cohérent avec la stack.
- ☐ La chaîne intègre toutes les étapes préalables (build).
- ☐ La chaîne exécute les tests automatiquement.
- ☐ Config versionnée sur Git.
- ☐ Documentation de la CI (install/config/test) rédigée et accessible.

C19. Livraison continue (CD)

- ☐ Les fichiers de config CD sont exécutés par le système.
- ☐ Les étapes de packaging (build, docker, minification) sont intégrées.
- ☐ L'étape de livraison (ex: Deploy ou Pull Request) est intégrée.
- ☐ Sources versionnées sur Git.
- ☐ Documentation de la CD (toutes étapes/déclencheurs) rédigée et accessible.

BLOC E5 : Maintenance et Monitoring

A9. Maintien en condition opérationnelle

C20. Surveiller l'application (Monitoring App)

- ☐ La documentation liste les métriques, seuils et valeurs d'alerte.
- ☐ La documentation justifie le choix des outils.
- ☐ Les outils (collecteurs, logs, dashboard) sont installés et opérationnels (minima en local).
- ☐ Les règles de journalisation (Logs) sont intégrées au code source.
- ☐ Les alertes sont configurées et fonctionnelles selon les seuils.
- ☐ Documentation d'installation du monitoring rédigée et accessible.

C21. Résoudre les incidents techniques

- ☐ La cause du problème est identifiée.
- ☐ Le problème est reproduit en environnement de dev.
- ☐ La procédure de débogage est documentée.
- ☐ La solution documentée explicite chaque étape de résolution.
- ☐ La solution (fix) est versionnée sur Git (ex: Merge Request).

Checklist des Ressources & Stack Technique à utiliser pour le projet

Pour pouvoir cocher toutes les cases ci-dessus, il faut impérativement avoir sélectionné et mis en place les outils suivants.

1. Gestion de Projet & Méthodologie

- ✓ Méthode Agile : Scrum
- ✓ Outil de gestion de tickets/Backlog : Github Projects
- ✓ Outil de communication : Discord
- ✓ Outil de wireframing/maquettage : Penpot
- ✓ Outil de modélisation données/flux : Looping

2. Développement & Code

- ✓ Gestionnaire de version (VCS) : Git
- ✓ Plateforme de dépôt distant : GitHub
- ✓ IDE / Éditeur de code : VSCode
- ✓ OS : Windows 11 Pro (PC Fixe et Portable)
- ✓ Terminal : Powershell (version la plus récente)
- ✓ CPU : AMD Ryzen 9 7950X (PC Fixe) et AMD Ryzen AI 9 HX 370 (PC Portable)
- ✓ RAM : 32 Gb DDR5 (PC Fixe et Portable)
- ✓ GPU : AMD 7900 XTX (PC Fixe) et chipset graphique pour PC Portable
- ✓ Gestionnaire de paquets/projet Python : uv (CLI Astral)
- ✓ Générateur de documentation : Sphinx
- ✓ Documentation en Markdown : MyST-Parser (extension Sphinx)
- ✓ Thème de documentation : Furo (thème Sphinx)

3. Data (Bloc E1)

- ✓ Langage de script de collecte : Python
- ✓ Librairie de requêtes HTTP : HTTPX
- ✓ Framework de crawl/scraping (HTTP, parsing, pipeline) : Scrapy
- ✓ Scraping de sites dynamiques (JS, rendu navigateur) : Playwright
- ✓ Intégration navigateur dans le crawler : scrapy-playwright

- ✓ Système de Gestion de Base de Données (SGBD - Relationnel) : PostgreSQL
- ✓ Système Big Data : Apache Spark (Traitement) + MinIO (Stockage Objet)
- ✓ Langage SQL : SQLAlchemy ORM 2.0 (Async)

4. Intelligence Artificielle (Blocs E2 & E3)

- ✓ Source de Veille IA : Inoreader + Task deep search auto Perplexity Pro (abonnement)
- ✓ Service IA Pré-existant (SaaS/PaaS) : HuggingFace Serverless Inference API
- ✓ Modèle IA (Code Custom) : Scikit-learn, TensorFlow, PyTorch, Hugging Face + AMD ROCm 7.1.1 (ou plus récent) version qui fonctionne nativement dans Windows
- ✓ Librairie de test IA : Deepchecks
- ✓ Outil de MLOps / Monitoring IA : MLFlow (suivi expériences) + Prometheus et Grafana (production)
- ✓ Outil de versionning de données : DVC (Data Version Control)

5. Backend & API

- ✓ Langage Backend : Python prioritaire mais autres possibles selon les outils choisis
- ✓ Framework API REST : FastAPI
- ✓ Outil de documentation API : Swagger UI + ReDoc
- ✓ Librairie de sécurité/Auth : FastAPI Users
- ✓ Bibliothèques et outils utiles : Ruff, Loguru

6. Frontend (Application cliente)

- ✓ Framework Frontend : React via build tool Vite
- ✓ Librairie UI / CSS : Shadcn/UI
- ✓ Outils d'accessibilité (Test) : axe-core via Playwright dans pipeline CI/CD

7. DevOps & Infrastructure (Blocs E4 & E5)

- ✓ Plateforme CI/CD : GitHub Actions
- ✓ Solution de Conteneurisation : Docker, Docker-compose
- ✓ Outil de Monitoring Applicatif (App & Logs) : Prometheus + Grafana + Loki (agrégation logs loguru)
- ✓ Hébergement / Déploiement (Pré-prod) : Render